

Hypabase Memory: A Hypergraph System for Long-Term Conversational Recall

Gamgee Labs Private Limited
<https://hypabase.app>

April 2026

Abstract

Agent memory is fundamentally a knowledge representation problem. Systems that store text chunks or binary triples are structurally limited: they discard the relational structure of facts, making precise retrieval dependent on embedding similarity alone. We present Hypabase Memory, a system that represents conversational knowledge as provenance-annotated hyperedges in a persistent hypergraph, extracts structured facts via Abstract Meaning Representation (AMR), and retrieves them using Query-Biased Personalized PageRank (QB-PPR) fused with semantic search. On LongMemEval, a 500-question benchmark for long-term conversational memory, Hypabase Memory achieves 87.4% accuracy (task-averaged 89.7%), including 100% on personalization tasks, with session recall@10 of 96.4%. All storage and retrieval runs locally in a single SQLite file. Our error analysis confirms the representational thesis: across all failure categories, errors trace to incomplete extraction rather than retrieval failure, indicating that representation quality is the primary determinant of system accuracy.

1 Introduction

Long-term memory for AI agents is widely treated as a retrieval problem: store conversation history, embed it, and fetch relevant passages at query time. We argue it is primarily a representation problem. The structure in which knowledge is stored determines the ceiling of what retrieval can achieve, regardless of the sophistication of the retrieval algorithm.

Current approaches to agent memory illustrate this. Retrieval-augmented generation over text chunks partitions conversation transcripts into overlapping segments and retrieves by cosine similarity. This is effective for verbatim recall but discards relational structure. A statement such as “I bought a bookshelf from a thrift store two weeks ago and am now repainting it” is stored as a single embedding vector. The buying event, its source, timing, and the repainting intent are conflated with no mechanism to query them independently.

Triple-based knowledge graphs recover some structure by extracting (subject, predicate, object) relations. But they constrain every fact to a binary relation. The statement above yields multiple disconnected triples: (*user*, *bought*, *bookshelf*), (*bookshelf*, *source*, *thrift store*), (*user*, *repainting*, *bookshelf*). The connection between these facts and their shared temporal context is severed. Aggregation, counting, and multi-hop reasoning over such fragments is unreliable.

Both approaches impose a structural ceiling. No retrieval algorithm can recover relational information that the representation discards at storage time.

Hypabase Memory is built on a different premise: facts are naturally N-ary, and the storage representation should preserve this. We represent conversational knowledge as hyperedges—single edges connecting two or more entities atomically, annotated with semantic roles, tem-

poral metadata, and provenance. The bookshelf example maps to one hyperedge connecting user, bookshelf, thrift store, two weeks ago, and repainting. The complete relational structure of the original statement is preserved in a single retrievable unit.

We extract knowledge using Abstract Meaning Representation (AMR), a formal semantic framework that maps natural language to predicate-argument structures [Banarescu et al., 2013]. We retrieve using Query-Biased Personalized PageRank on the hypergraph fused with semantic embedding search via Reciprocal Rank Fusion. The complete system, including graph storage, vector index, and full-text search, operates within a single SQLite file with no external service dependencies.

On LongMemEval [Wang et al., 2025], Hypabase Memory achieves 87.4% overall accuracy, including 100% on personalization tasks and session recall@10 of 96.4%. Our error analysis validates the representational thesis: across all failure categories, retrieval succeeds in surfacing relevant memories with near-perfect recall. Where the system fails, the cause is incomplete or imprecise knowledge extraction—the upstream representation, not the downstream retrieval. This confirms that improvements to the representation layer (better extraction, hierarchical summarization) have the highest expected impact on overall accuracy.

2 Design Principles

Three principles guide the design of Hypabase Memory. Each addresses a structural limitation of existing systems and reflects our position that representation quality determines the performance ceiling.

2.1 Hypergraph Representation

A knowledge graph edge connects exactly two nodes. A hyperedge connects an arbitrary number of nodes. This distinction is significant for memory systems because real-world facts are rarely binary.

The sentence “Bob had lunch with Carol at the diner on Tuesday for \$25” produces, under triple-based extraction, five separate relations:

```
(Bob, had_lunch_with, Carol)
(Bob, ate_at, diner)
(lunch, occurred_on, Tuesday)
(lunch, cost, $25)
(Bob, had, lunch)
```

These triples are individually retrievable but no longer form a coherent event. A query such as “How much did Bob spend at the diner?” requires the system to locate the cost triple and join it with the correct lunch event across fragmented relations.

A hyperedge preserves the event as a single unit:

```
(dined :subject Bob :instrument Carol :locus "diner"
      :locus Tuesday :value "$25" :tense past
      :memory_type episodic)
```

This representation connects Bob, Carol, the diner, Tuesday, and \$25 in one edge. Retrieving any constituent entity in the context of a dining-related query returns the complete event with all details intact.

The structural consequence for retrieval is direct: a hyperedge concentrates relevance mass on entities that co-participate in a fact. In a triple-based graph, equivalent mass is distributed across disconnected edges, reducing the effectiveness of graph-based traversal. This property is central to graph-based retrieval; it determines whether structural traversal algorithms can identify coherent facts at all.

Every hyperedge additionally carries provenance metadata: a source identifier (the conversation session from which the fact was extracted) and a confidence score. When user preferences or facts change over time, the system surfaces the most recently recorded value with full provenance, rather than silently discarding history.

2.2 Structured Extraction via AMR

Most retrieval-augmented systems treat text as the unit of storage. Hypabase Memory extracts structured representations from text, storing facts rather than passages.

We use Abstract Meaning Representation (AMR), a formalism from computational linguistics [Banarescu et al., 2013] that represents sentence meaning as a rooted, directed graph of concepts and semantic roles. We serialize AMR graphs in PENMAN notation, a compact S-expression format.

AMR offers three properties that are well-suited to memory extraction.

First, it defines a principled role system. AMR specifies a fixed set of semantic roles (agent, patient, instrument, location, time) grounded in linguistic theory. We map these to *kāraka* roles from Pāṇinian grammar, providing consistent semantic labeling across all extracted facts.

Second, it enforces a one-sentence-one-graph correspondence. This design principle naturally produces the dense, multi-role hyperedges that the system requires: all details of an event are captured in one structure, rather than fragmented into sparse triples.

Third, PENMAN notation has a defined grammar, making LLM outputs parseable and validatable. Malformed extractions are caught at parse time rather than propagating silently into the knowledge base.

We use an LLM (Claude Sonnet 4.6) for AMR extraction rather than a trained parser. This trades parsing throughput for flexibility with informal language, implicit references, and domain-specific terminology. The extraction prompt enforces AMR conventions: dense graphs, resolved pronouns, canonical entity names, and preserved specifics (numbers, dates, names).

Conversation sessions are processed in non-overlapping windows of three exchange pairs. Each window is processed in a single LLM call. This eliminates the duplicate extraction characteristic of sliding-window approaches, where the same exchange appears in multiple overlapping contexts, while preserving coreference resolution within each group.

2.3 Dual-Arm Retrieval

The hypergraph representation enables retrieval strategies that are unavailable to flat or triple-based stores. Once facts are stored as hyperedges in a graph, structural traversal becomes possible alongside semantic search.

Two facts may be semantically distant in embedding space yet structurally connected through shared entities in the hypergraph. A user who discusses hybrid bike gear maintenance in one session and road trip mileage in another produces memories in different embedding neighborhoods. A query about “bike expenses” should retrieve both, but cosine similarity between the query and the maintenance-focused memories may fall below the retrieval threshold. In

the hypergraph, both facts are connected through the “hybrid bike” entity node, making them reachable via graph traversal.

Our retrieval combines two complementary arms. The structural arm runs Personalized PageRank on the hypergraph, seeded at query-relevant nodes, with inverse-degree vertex weighting to suppress hub nodes. The semantic arm performs cosine KNN on edge embeddings. The two arms are fused via Reciprocal Rank Fusion (RRF), which combines rankings without requiring score calibration. The structural arm surfaces entity-connected memories that embeddings miss; the semantic arm surfaces thematically similar memories that lack direct graph connections.

This dual-arm design is enabled by the hypergraph representation, which preserves entity-level graph structure absent from text chunk and triple-based stores.

3 Architecture

Hypabase Memory is implemented as a Python library with all persistence in SQLite. The system operates in four phases: ingestion, consolidation, recall, and answering.

3.1 Storage Layer

All data resides in a single SQLite file containing seven core tables and two virtual tables:

- **meta**: key-value storage for schema version and configuration.
- **nodes** and **edges**: the hypergraph structure. Edges connect N nodes via an **incidences** junction table that preserves node ordering.
- **vertex_set_index**: SHA-256 hash index for $O(1)$ exact vertex-set lookup, used for duplicate detection.
- **embeddings**: text and binary float32 embedding data for both nodes and edges.
- **access_log**: memory access tracking for strength-based decay models.
- **edge_fts**: FTS5 virtual table providing BM25 full-text search over edge content.
- **vec_embeddings**: sqlite-vec virtual table providing KNN search over embedding vectors using cosine distance.

SQLite operates in WAL mode with foreign keys enabled. The graph supports namespace isolation, temporal validity (`valid_at/expired_at` timestamps for point-in-time queries), and soft deletes.

3.2 Ingestion Pipeline

Ingestion transforms conversation sessions into hypergraph structure through five stages.

Windowing. Sessions are split into exchange pairs (user message and assistant response), then grouped into non-overlapping windows of three exchanges.

AMR extraction. Each window is processed by an LLM with a system prompt enforcing AMR conventions: dense multi-role graphs, resolved pronouns, canonical entity names, and preserved specifics. The LLM outputs PENMAN notation.

Parsing. PENMAN output is parsed into structured atoms consisting of a verb, semantic roles, modifiers, and contexts. The parser supports nested graphs for complex statements.

Entity resolution. Entity names are normalized (lowercased, whitespace-collapsed) and resolved against a cache of known entities. New entities are registered as graph nodes and embedded for future retrieval.

Edge creation. Each parsed atom becomes a hyperedge connecting its entity nodes, with properties for memory type (episodic, semantic, procedural), mood (actual, planned, uncertain), tense, negation, and importance. The edge text, a natural language rendering of the atom, is embedded for semantic search.

3.3 Consolidation

After ingestion, a consolidation pass merges duplicate entities through four steps. All entity nodes are embedded. Pairwise cosine similarity is computed on node embeddings. Nodes with similarity ≥ 0.95 are grouped into connected components. The highest-degree node in each component is selected as the canonical representative. Infrastructure edges of type `same_as` link aliases to canonical nodes, and the entity resolver cache is updated so that future queries resolve aliases transparently.

Consolidation is a batch operation executed once after all sessions are ingested. This design follows the principle, articulated by [Stewart and Buehler \[2026\]](#), of separating extraction (write-time) from deduplication (consolidation-time), avoiding the complexity of online merge decisions during ingestion.

3.4 QB-PPR Recall

Query-Biased Personalized PageRank (QB-PPR) is the primary retrieval algorithm. It adapts the hypergraph random walk model of [Chitra and Raphael \[2019\]](#) with three corrections for the star topology characteristic of personal memory graphs.

Topology. Personal memory graphs exhibit a characteristic star structure: a central “user” node is connected to nearly every edge, with specific entities (people, places, events) at the periphery. Standard PageRank assigns most mass to the hub node, which provides no discriminative signal. The corrections below concentrate mass on specific, query-relevant entities.

Inverse-degree vertex weights. Within each hyperedge, nodes are weighted inversely by their global degree:

$$\gamma_e(v) = \frac{1 / \deg(v)}{\sum_{u \in e} 1 / \deg(u)} \quad (1)$$

where $\deg(v)$ is the global degree of node v across the full graph, and the normalization sum is computed over nodes participating in edge e . On a representative memory graph, the “user” node (degree ~ 150) receives $\gamma \approx 0.004$, while a specific entity such as “hybrid bike” (degree 2) receives $\gamma \approx 0.3$. The random walker preferentially transitions to specific, low-degree nodes.

Non-linear edge weight amplification. Edges are weighted by their cosine similarity to the query, raised to a power:

$$\omega_q(e) = \cos(e, q)^\beta, \quad \beta = 4 \quad (2)$$

The fourth power maps a 1.3:1 ratio between top and second-ranked cosine similarities (e.g., 0.65 vs. 0.50) to approximately 3:1 in edge weights. This amplification produces meaningful walk bias on the small graphs (50–200 edges) typical of personal memory.

High teleportation rate. We set $\alpha = 0.5$ (50% teleportation probability) with 5 iterations. On small graphs, lower teleportation rates cause the walk to converge toward the stationary distribution, attenuating the query-biased signal. A high teleportation rate maintains mass near the seed nodes, which are the entities identified as query-relevant via embedding similarity.

Seed selection. Individual query words are embedded and matched to graph nodes (`min_score` = 0.65). The full query string is also matched (`min_score` = 0.5, `limit` = 8). The union of all matched nodes forms the teleportation target set.

Edge scoring. After convergence, each hyperedge is scored as the weighted sum of its constituent nodes' PageRank mass:

$$\text{score}(e) = \sum_{v \in e} \pi(v) \cdot \gamma_e(v) \quad (3)$$

Fusion. PPR-scored edges and cosine KNN results (top 150 candidates, $\text{min_score} = 0.3$) are combined via Reciprocal Rank Fusion:

$$\text{RRF}(e) = \frac{1}{k + \text{rank}_{\text{PPR}}(e)} + \frac{1}{k + \text{rank}_{\text{cos}}(e)}, \quad k = 60 \quad (4)$$

The top 50 edges by RRF score are returned as memory context for the answering model.

4 Theoretical Foundations

The design of Hypabase Memory is grounded in established research across graph theory, cognitive science, linguistics, and knowledge representation. We describe each foundation and the specific design decision it informs.

4.1 Hypergraph Random Walks

[Chitra and Raphael \[2019\]](#) extend standard graph random walks to hypergraphs by defining a two-step transition: from a node, the walker selects an incident hyperedge (weighted by edge weight), then selects a destination node within that edge (weighted by vertex weights γ). The authors prove convergence to a unique stationary distribution under regularity conditions on the weight functions.

QB-PPR implements this framework with inverse-degree γ and cosine-based edge weights (Section 3.4). The vertex weight function within each edge enables encoding of structural preferences, specifically hub suppression and specificity amplification, that standard PageRank on pairwise graphs cannot express. This is a capability unique to hypergraph-based retrieval: it requires both N-ary edges and per-node-per-edge weight functions.

4.2 Complementary Learning Systems

CLS theory [[McClelland et al., 1995](#)] proposes that the brain employs two complementary systems for memory: the hippocampus for rapid encoding of specific episodes, and the neocortex for gradual extraction of statistical regularities. Retrieval involves interaction between these systems, with the hippocampus providing specific indices into neocortical distributed representations.

Our dual-arm retrieval architecture reflects this model. The structural arm (PPR on the hypergraph) functions as a hippocampal index, traversing entity connections to locate specific related memories. The semantic arm (embedding KNN) functions as neocortical pattern completion, identifying memories that are distributionally similar to the query. RRF fusion integrates signals from both pathways. The hypergraph representation is what makes the hippocampal arm possible; systems that store only embeddings have access to only one of these two retrieval pathways.

4.3 ACT-R Memory Model

The ACT-R declarative memory module [[Anderson et al., 2004](#)] defines activation as a function of base-level learning (recency and frequency) and spreading activation from cues. The

fan effect describes how each cue’s activation is divided among its associated items, causing specific cues to be more effective than general ones.

Inverse-degree vertex weighting in QB-PPR is a graph-theoretic analog of the fan effect: nodes connected to many edges (high fan) contribute less activation per edge, while nodes with few connections (low fan) contribute more. The production recall pipeline (`Memory.recall()`) directly implements ACT-R activation scoring with exponential recency decay and logarithmic frequency scaling.

4.4 Higher-Order Knowledge Representation

Stewart and Buehler [2026] propose representing knowledge as higher-order relations (hyperedges) rather than binary triples, with provenance metadata attached to each relation. Their design principles include zero-overlap chunking at extraction time to prevent duplicate extraction, batch consolidation rather than online deduplication, and representation of N-ary events as single hyperedges.

Hypabase Memory implements all three principles: adjacent-window extraction with no overlap, post-ingestion consolidation via cosine-based entity merging, and AMR-to-hyperedge mapping that preserves multi-participant events as atomic edges.

4.5 Abstract Meaning Representation

AMR [Banarescu et al., 2013] is a semantic representation language that maps sentences to rooted, directed, labeled graphs. Its design goals of abstracting from surface syntax, normalizing semantic roles, and producing one graph per sentence align with the requirements for dense, canonical fact representations. We use AMR’s PENMAN serialization as the extraction target, providing a parseable and validatable output format.

Our mapping from AMR roles to *kāraka* roles (from Pāṇinian Sanskrit grammar) provides an alternative semantic labeling system. While *kāraka* roles are arguably more language-universal than AMR’s English-centric labels, in practice both serve the same function: consistent annotation of participant roles across extracted facts.

4.6 Hypergraph Knowledge Representation

The use of hypergraphs for knowledge representation has precedent in artificial general intelligence research, notably OpenCog’s AtomSpace [Goertzel et al., 2014], which represents knowledge as a metagraph with typed nodes and links of arbitrary arity. Hypabase Memory targets a narrower problem than general knowledge representation, but the architectural premise is shared: knowledge is naturally higher-order, and compressing it into binary relations discards information that downstream reasoning requires.

5 Evaluation

5.1 LongMemEval Benchmark

LongMemEval [Wang et al., 2025] evaluates long-term memory in conversational AI systems. It consists of 500 questions across six categories, each targeting a distinct memory capability (Table 1).

Each question provides haystack sessions (conversation history to ingest) and a question with a ground-truth answer. Correctness is determined by an LLM judge using official per-type prompts from the benchmark, producing a binary verdict.

Table 1: LongMemEval question categories.

Category	Capability tested	<i>N</i>
Single-session user	Recall of facts stated by the user	70
Single-session assistant	Recall of facts stated by the assistant	56
Single-session preference	Application of user preferences	30
Multi-session	Aggregation across multiple sessions	133
Temporal reasoning	Ordering and duration of events	133
Knowledge update	Most recent value when facts change	78

We use the oracle variant, which provides the minimal set of sessions required to answer each question. This isolates memory system performance from distractor-filtering ability.

5.2 Configuration

Table 2: Evaluation configuration.

Component	Setting
Ingestion model	Claude Sonnet 4.6 (AWS Bedrock)
Answering model	Claude Opus 4.6 (AWS Bedrock)
Judge model	Claude Opus 4.6 (AWS Bedrock)
Embedding model	BAAI/bge-small-en-v1.5 (384 dims, FastEmbed/ONNX)
Ingestion window	3 exchange pairs, non-overlapping
Recall algorithm	QB-PPR with RRF fusion
Recall limit	50 memories
PPR parameters	$\alpha = 0.5$, $\beta = 4.0$, iterations = 5
RRF parameter	$k = 60$

The full benchmark (500 questions including ingestion, consolidation, recall, answering, and judging) completed in 647 seconds with 5 parallel workers, averaging 1.3 seconds per question end-to-end. LLM calls dominate wall-clock time; local recall averages under 0.3 seconds per query on graphs of 50–200 edges.

5.3 Results

Table 3: LongMemEval results by category.

Category	<i>N</i>	Correct	Accuracy
Single-session preference	30	30	100.0%
Single-session user	70	65	92.9%
Single-session assistant	56	50	89.3%
Knowledge update	78	69	88.5%
Temporal reasoning	133	116	87.2%
Multi-session	133	107	80.5%
Overall	500	437	87.4%

Task-averaged accuracy (mean of per-category accuracies): **89.7%**.

The system achieves 100% accuracy on single-session preference. These questions require the system to recall user preferences from prior conversations and apply them to new situations—

the core use case for personalized AI agents. The hypergraph representation preserves the preference, its context, and its provenance as a single edge, enabling reliable retrieval even when the new situation uses different language than the original conversation.

5.4 Retrieval Quality

We report retrieval quality independently of the LLM judge using session-level recall_all@k : the fraction of questions for which all relevant sessions are represented in the top- k retrieved memories. In the oracle setting, all provided sessions are relevant by definition, so session-level precision is 1.0 and NDCG is 1.0 at all cutoffs. Recall_all@k measures how much retrieval depth is needed to achieve full coverage.

Table 4: Session-level recall_all@k by category.

k	Overall	SS-U	SS-A / SS-P / KU	MS	TR
10	94.2%	100%	100%	90.2%	88.0%
20	98.6%	100%	100%	98.5%	96.2%
30	99.8%	100%	100%	99.2%	100%
50	100%	100%	100%	100%	100%

Task-averaged recall_all@10 is 96.4%. All single-session and knowledge-update categories achieve perfect recall at $k = 10$, requiring only one or two sessions. Multi-session and temporal reasoning questions, which span more sessions, require greater retrieval depth: 90.2% and 88.0% at $k = 10$, converging to 100% by $k = 40$.

These results support the hypothesis advanced in Section 1. The retrieval system ranks relevant memories with high priority across all categories. When end-to-end accuracy falls below 100%, the cause is not retrieval failure but limitations in the upstream representation (Section 6).

5.5 Ablation: Recall Algorithm

Table 5: Recall algorithm comparison.

Recall method	Accuracy
Memory.recall() (CLS interleave, limit = 30)	86.4%
QB-PPR + RRF (limit = 50)	87.4%

QB-PPR provides a 1.0 percentage point improvement over the production CLS-based recall. The gains are concentrated in temporal reasoning (+2.3%) and multi-session (+1.8%) categories, where graph-structural connections between entities across sessions surface relevant memories that embedding similarity alone does not rank sufficiently.

5.6 Ablation: Recall Limit

An adaptive limit that scaled with graph size (retrieving 51–80 memories for larger graphs) severely degraded accuracy. Additional context introduces noise that overwhelms the answering model, particularly for temporal reasoning and knowledge-update questions where the model must identify the most recent or correctly ordered facts among many candidates. The fixed limit of 50 represents the optimal tradeoff between coverage and precision in our evaluation.

Table 6: Recall limit sensitivity.

Recall method	Limit	Accuracy
Memory.recall() (baseline)	30	86.4%
QB-PPR + RRF	50	87.4%
QB-PPR + RRF (adaptive)	51–80	66.0%

6 Error Analysis

Our analysis of failure modes provides evidence for the representational hypothesis: retrieval consistently surfaces relevant memories, and the remaining errors trace to information that was not stored or not stored with sufficient precision.

6.1 Multi-Session Aggregation

Multi-session questions requiring counting or aggregation achieve 80.5% accuracy. The failure mode is systematic: each session’s facts are extracted independently by separate LLM calls, producing atoms with varying wording, levels of detail, and entity naming conventions. The answering model must then count across these scattered, inconsistently described atoms, a task that requires distinguishing distinct events from the same event described differently.

This is a representation problem rather than a retrieval problem. The hypergraph contains the relevant facts (retrieval recall is near-perfect), but the representation does not make aggregation straightforward. Hierarchical summarization—maintaining explicit counts and aggregates at a summary level—is the anticipated solution (Section 7.3).

6.2 Temporal Ordering

Temporal reasoning questions achieve 87.2% accuracy. Failures concentrate in cases where ingested memories contain only relative time references (“last week,” “two months ago”) without anchoring to absolute dates. The system records session dates as provenance metadata, but AMR extraction does not consistently resolve relative references to absolute timestamps. When two events are both described as occurring “recently” in different sessions, the answering model cannot determine their ordering from memory content alone.

This is again an extraction problem: the temporal information exists in the source conversations but is not captured with sufficient precision during ingestion. Temporal validity modeling (Section 7.4) addresses this.

6.3 Ingestion Quality as Performance Ceiling

Across all failure categories, we consistently observe that retrieval succeeds in surfacing relevant memories (97.7% turn-level coverage, 96.4% session recall@10), but the memories themselves are incomplete or imprecise. The LLM extraction drops specific numbers, rounds quantitative values, substitutes vague descriptions for stated specifics, and occasionally misattributes facts across entities. Each such ingestion error is unrecoverable: no retrieval algorithm, reranker, or answering strategy can compensate for information that was never stored.

This finding reinforces the central argument of this paper: agent memory is a representation problem. The retrieval system achieves near-perfect recall across all categories on this benchmark. The performance ceiling is determined by what gets represented in the graph, not by how it is retrieved. Further accuracy improvements depend primarily on extraction quality:

improved prompts, specialized parsers, verification passes, and richer representational structures such as hierarchical summaries.

7 Future Directions

7.1 Cross-Encoder Reranking

A second-stage cross-encoder applied after QB-PPR retrieval would jointly score each retrieved memory against the query. Based on published results in the retrieval literature, cross-encoder reranking typically recovers 3–5 percentage points on precision-sensitive tasks such as temporal reasoning and knowledge update. We consider this the highest-impact improvement available without changes to the representation layer.

7.2 Dedicated AMR Parsers

Our current ingestion uses a general-purpose LLM for AMR extraction. Dedicated AMR parsers such as AMRBART [Bai et al., 2022] and spring [Bevilacqua et al., 2021] are faster, less expensive, and produce more consistent output. The tradeoff is flexibility: trained parsers perform well on standard English but may degrade on informal conversational language, code-switching, and domain-specific terminology. A hybrid approach using a trained parser for standard cases and an LLM for edge cases could reduce ingestion cost by an order of magnitude while maintaining quality.

7.3 Hierarchical Summarization

The system currently stores only episodic memories (individual facts from individual conversations). Adding entity-level summaries and theme-level summaries would directly address the multi-session aggregation weakness identified in Section 6.1.

The principal design challenge is maintaining consistency between the episodic layer and the summary layer as new information arrives. Summaries must be updated incrementally, not regenerated from scratch, and contradictions between episodic memories and summaries must be resolved explicitly. The hypergraph representation provides a natural substrate for this: summaries can be stored as higher-level edges that reference the episodic edges they aggregate, with provenance chains maintaining traceability.

7.4 Temporal Validity Modeling

Knowledge-update questions require distinguishing current facts from superseded ones. The system stores temporal metadata (`created_at` timestamps) but does not yet implement formal temporal validity: marking outdated values as expired when new values are ingested. Adding `valid_at`/`expired_at` semantics to the ingestion pipeline would enable the retrieval system to prefer the most recent value automatically.

7.5 Memory Strength Calibration

The production recall pipeline includes an ACT-R-inspired memory strength model with exponential recency decay, logarithmic frequency scaling, and per-type decay rates (episodic memories decay faster than semantic ones). The benchmark evaluation bypasses this model because all memories are ingested simultaneously. In production deployment, where memories accumulate over weeks and months, calibrating the strength parameters against observed usage patterns will be necessary for balancing recency against historical context.

8 Conclusion

We set out to test the hypothesis that agent memory is primarily a representation problem, and that hypergraph-based knowledge representation provides a stronger foundation for long-term conversational recall than text chunks or binary triples. The results support this hypothesis.

On LongMemEval, Hypabase Memory achieves 87.4% accuracy (task-averaged 89.7%) with 100% on personalization tasks and session recall@10 of 96.4%. The system operates entirely locally in a single SQLite file, with no external service dependencies. The hypergraph is queryable, visualizable, and fully inspectable.

The error analysis provides the most informative evidence for the representational thesis. Across all failure categories, retrieval succeeds: the graph surfaces relevant memories with near-perfect recall. Where the system falls short, the cause is incomplete knowledge extraction—information that was present in the source conversation but not captured with sufficient precision during ingestion. This confirms that the highest-leverage path to improved accuracy is in the representation layer: better extraction, richer structures, hierarchical summarization.

Hypergraphs with provenance, AMR-based extraction, and graph-aware retrieval are not a complete solution to agent memory. However, they provide a structured, inspectable, and extensible foundation on which more sophisticated capabilities—reranking, summarization, temporal reasoning, and production-calibrated memory decay—can be systematically built.

References

- Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., and Qin, Y. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.
- Bai, X., Chen, Y., and Song, L. Graph pre-training for AMR parsing and generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- Banarescu, L., Bonial, C., Cai, S., Georgescu, M., Griffitt, K., Hermjakob, U., Knight, K., Koehn, P., Palmer, M., and Schneider, N. Abstract Meaning Representation for sembanking. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, 2013.
- Bevilacqua, M., Blloshmi, R., and Navigli, R. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- Chitra, U. and Raphael, B. J. Random walks on hypergraphs with edge-dependent vertex weights. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.
- Goertzel, B., Pennachin, C., and Geisweiller, N. *Engineering General Intelligence, Part 2: The CogPrime Architecture for Integrative, Embodied AGI*. Springer, 2014.
- McClelland, J. L., McNaughton, B. L., and O’Reilly, R. C. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.
- Stewart, T. and Buehler, M. Higher-order knowledge representation for AI agent memory. *Preprint*, 2026.
- Wang, D., et al. LongMemEval: Benchmarking long-term memory in conversational AI. In *International Conference on Learning Representations (ICLR)*, 2025.